CIE-1 STUDY GUIDE · UNIT 3 OF 3 · CS51

Unit 3
Architecture

SOFTWARE ENGINEERING & MODELLING

The big-picture structure of a system: the decisions architects make and the different views used to describe an architecture.

IN THIS UNIT

Chapter 6 · Architectural Design: Decisions and Views

SYLLABUS

Architectural Design: Architectural design decisions, Architectural views, ~~Architectural patterns.~~

~~**Design and implementation:** Object-oriented design using the UML, Design patterns, Implementation issues.~~

Built from **your lecture slides**, **Sommerville (10th ed)** and **Pressman**.

Course coordinators
Mr. Pradeep Kumar D · Dr Shilpa S Chaudhari

Start here: how to use this guide

This guide covers **exactly the CIE-1 portion** of Software Engineering & Modelling (CS51): all of Unit 1, all of Unit 2, and Unit 3 up to **architectural views**. It assumes you know nothing yet. Every point from your lecture slides is here, explained in plain English first, then in the textbook wording you should use in answers.

Unit	Syllabus topics (CIE-1)	Where in this guide
Unit 1	**Introduction:** professional software development, software engineering ethics	Chapter 1 · Introduction
Unit 1	**Software processes:** process models, process activities, coping with change, process improvement	Chapter 2 · Software Processes
Unit 1	**Agile software development:** agile methods, agile development techniques, agile project management	Chapter 3 · Agile Development
Unit 2	**Requirements engineering:** functional & non-functional requirements, RE processes, elicitation, specification, validation, change	Chapter 4 · Requirements Engineering
Unit 2	**Design concepts:** design within the context of SE, design concepts, the design model	Chapter 5 · Design Concepts
Unit 3	**Architectural design:** architectural design decisions, architectural views (patterns onwards are after CIE-1)	Chapter 6 · Architectural Design

How each chapter works

1. **Opener page**: what the chapter covers, how long it takes and what you will be able to do.
2. **Sections**: each idea is explained simply, then given in exam wording, with an example and a diagram where one helps.
3. **Chapter in one page**: a dense revision sheet. Re-read it the night before the exam.
4. **Likely exam questions**: each one says what to write and which diagram to draw.

What the coloured boxes mean

DEFINITION — LEARN THIS LINE
A definition to learn **word for word**. Write it as the first line of your answer.

★ KEY POINT
A fact you must remember. These are the things examiners look for.

IN SIMPLE WORDS
The idea in everyday language. Read this first if a topic feels confusing.

EXAMPLE
A worked example, usually from the textbook case studies.

✎ EXAM TIP
How to write the answer: structure, diagram, keywords.

⚠ WATCH OUT
A common confusion or mistake that loses marks.

MEMORY TRICK
A trick for remembering a list.

? QUICK CHECK
Test yourself. Cover the answer and try it first.

Highlighted phrases are the exact keywords to use in your answers.

Priority badges

★★★ MUST KNOW

Asked often: definitions, models, diagrams, comparisons. Study these first and practise the diagrams.

★★ IMPORTANT

Commonly asked as short questions or parts of long answers. Understand them well.

★ READ ONCE

Background or lower-priority material. Read it once so nothing surprises you.

Study plans

If you have a week

- **Day 1**: Ch 1 + start Ch 2
- **Day 2**: finish Ch 2 and draw every model
- **Day 3**: Ch 3
- **Day 4**: Ch 4
- **Day 5**: Ch 5 + Ch 6
- **Day 6**: all question banks
- **Day 7**: one-page recaps + diagrams

If you have 3 days

- **Day 1**: Ch 1–3, focusing on ★★★ MUST KNOW
- **Day 2**: Ch 4–6, focusing on ★★★ MUST KNOW
- **Day 3**: question banks, then recaps; redraw every diagram once

If it's the night before

- Read all six **one-page recaps**
- Redraw the must-know diagrams from memory
- Skim each question bank and say the answer outline out loud
- Sleep; recall is better when you're rested

✎ GOLDEN RULE FOR SE ANSWERS

For anything worth more than 2 marks: **definition → diagram → explanation in points → example → advantages/disadvantages (or comparison)**. A neat, labelled diagram earns marks even when your text is short.

⚠ NOT IN CIE-1, SO SKIP THESE FOR NOW

Unit 3 from **architectural patterns** onwards (MVC, layered, repository, client–server, pipe-and-filter, application architectures), object-oriented design using UML, design patterns and implementation issues. Also all of **Unit 4** (testing, evolution) and **Unit 5** (project management, planning, quality). They come after CIE-1.

Contents · Unit 3

UNIT 3 **Architectural Design (CIE–1 part)** 4

CH 6 **Architectural Design: Decisions and Views** 5

6.1 What is architectural design? 5
6.2 Example: a packing robot control system 7
6.3 Architectural abstraction: small and large 7
6.4 Advantages of explicit architecture 8
6.5 Architectural representations and their use 8
6.6 Architectural design decisions 10
6.7 Architecture reuse 11
6.8 Architecture and system characteristics 11
6.9 Architectural views 12
6.10 Representing architectural views 14
Chapter 6 in one page 15
Likely exam questions 16

EXAM **Exam-Day Playbook** 17

CIE-1 · UNIT 3

Unit 3: Architectural Design (CIE-1 part)

3

The big-picture structure of a system: the decisions architects make and the different views used to describe an architecture.

SYLLABUS, WORD FOR WORD

Architectural Design: Architectural design decisions, Architectural views, ~~Architectural patterns.~~

~~**Design and implementation:** Object-oriented design using the UML, Design patterns, Implementation issues.~~

CHAPTERS IN THIS UNIT

CHAPTER 6

Architectural Design: Decisions and Views

Box-and-line models · Design decisions · Non-functional requirements · 4+1 views

page 5

COURSE OUTCOME

CO3: Develop architectural and detailed design using UML and design patterns.

LAB COMPONENT

High level and low level design of the analysed real-time case study application.

CIE-1 SCOPE

Only up to architectural views is in CIE-1. Struck-through topics come later.

UNIT 3 · CHAPTER 6

Architectural Design: Decisions and Views

Box-and-line models · Design decisions · Non-functional requirements · 4+1 views

STUDY TIME
About 2 hours

SOURCES
Your Ch 6 slides (1–20) + Sommerville Ch 6 (introduction, 6.1, 6.2)

EXAM WEIGHT
The Unit 3 part of CIE-1: short definitions plus one diagram answer (design decisions or 4+1 views)

AFTER THIS CHAPTER YOU CAN

- Define architectural design and its output
- Explain why architecture comes first, even in agile
- Draw and explain the packing robot architecture
- Contrast architecture in the small and large
- List three advantages of explicit architecture
- Discuss box-and-line diagrams and model uses
- Draw the architectural design decisions
- Match non-functional requirements to architecture
- Draw and explain the 4+1 view model
- Compare UML, ADLs and informal notations

6.1 What is architectural design? ★★★ MUST KNOW

Before builders lay a brick, an architect decides the overall layout of the building. Software needs the same big-picture plan. The **software architecture** of a system is its overall organization: the main parts (**components** or **sub-systems**) and the relationships between them. **Architectural design** produces this plan.

> **DEFINITION — LEARN THIS LINE**
>
> **Architectural design** — is concerned with understanding how a software system should be organized and designing the overall structure of that system.

Three points to write in any answer (slide 3):

1. **What it does** — works out how the system should be organized and designs its overall structure.
2. **Where it sits** — it is the first stage of the software design process and the critical link between design and requirements engineering, because it identifies the main structural components in a system and the relationships between them.
3. **What it produces** — an **architectural model** that describes how the system is organized as a set of communicating components.

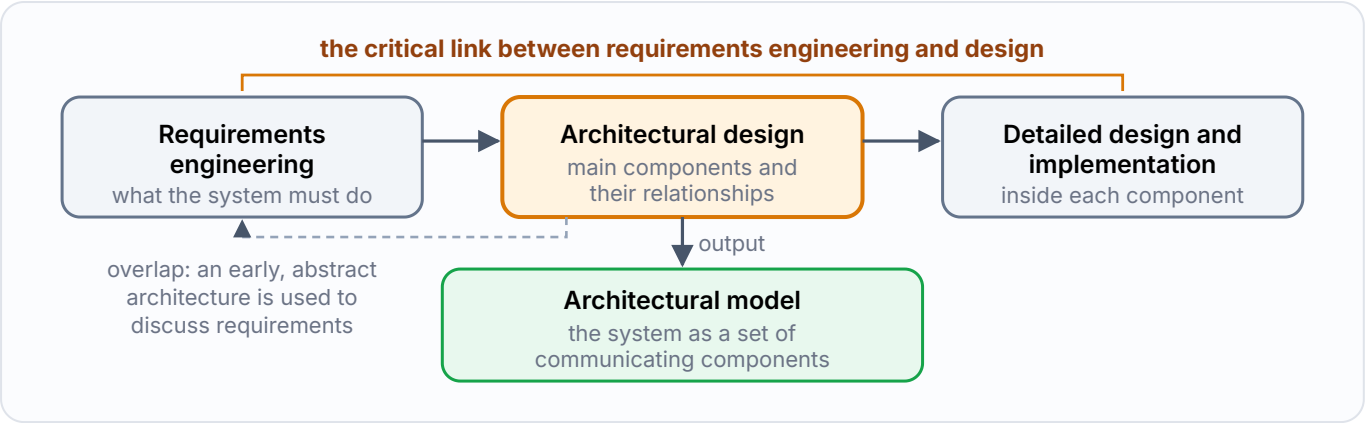


Fig 6.1 Where architectural design sits: between knowing *what* the system must do and designing the inside of each part. Drawn for this guide

Component	A part of the system with one well-defined job (e.g. the "vision system" of a robot). Large components are called sub-systems.
Stakeholder	Anyone with an interest in the system: users, customers, managers, engineers.
Functional / non-functional	Functional requirements say *what* the system does; non-functional requirements say *how well* (performance, security, safety, availability, maintainability).

Requirements and architecture overlap ★ READ ONCE

Ideally a specification contains no design, but this is unrealistic except for very small systems. In practice **requirements engineering and architectural design overlap**: you may propose an abstract architecture that groups system functions or features into large-scale components or sub-systems, and use it to discuss the requirements with stakeholders.

Why architecture matters ★★★ MUST KNOW

Software architecture affects the **performance, robustness, distributability and maintainability** of a system (Bosch). Individual components implement the functional requirements, but the dominant influence on the non-functional characteristics is the system's architecture. A study of "architecturally significant requirements" (Chen et al.) confirmed that non-functional requirements have the most significant effect on a system's architecture.

Agility and architecture ★★★ MUST KNOW

Agile methods build software in small increments with little design done up front. Even so:

- It is generally accepted that an **early stage** of agile processes is to design an overall system architecture. Incremental development of architectures is not usually successful.
- **Refactoring** (restructuring software to improve it without changing what it does) individual *components* in response to change is relatively easy.
- Refactoring the system *architecture* is usually **expensive because it affects so many components**: you may need to modify most system components to adapt them to the change.

> **IN SIMPLE WORDS**
> Repainting a room is cheap; moving the load-bearing walls means rebuilding most of the house. Components are the rooms; the architecture is the load-bearing walls.

6.2 Example: a packing robot control system ★★★ MUST KNOW

Sommerville's example is a **packing robot** that can pack different kinds of objects. It uses a vision component to pick out objects on a conveyor, identify the type of object and select the right kind of packaging. It then moves objects from the delivery conveyor to be packaged, and places packaged objects on another conveyor. Fig 6.2 is an **abstract model**: it shows only these components and the links between them.

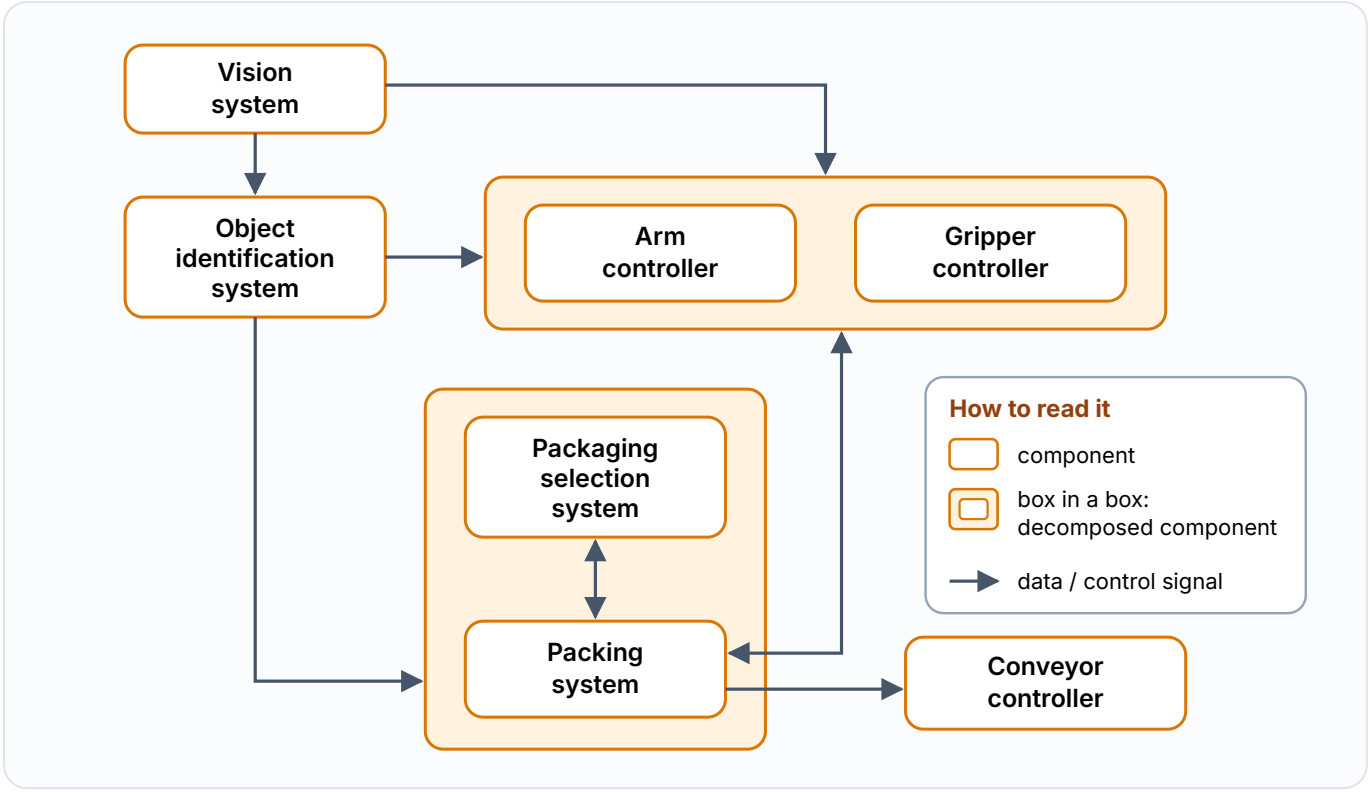


Fig 6.2 The architecture of a packing robot control system. Sommerville Fig 6.1 · Your slides, slide 5

How to read such a **block diagram** (also called a **box-and-line diagram**):

- Each **box** represents a component.
- **Boxes within boxes** mean the component has been decomposed into sub-components (Arm + Gripper controllers form one sub-system; Packaging selection + Packing form another).
- **Arrows** mean that data and/or control signals pass from component to component in the direction of the arrow. When you redraw Fig 6.2, keep the arrow directions and the two shaded groups.

6.3 Architectural abstraction: small and large ★★ IMPORTANT

Architectures can be designed at two levels of abstraction (amount of detail):

- **Architecture in the small** is concerned with the architecture of **individual programs**. At this level, we are concerned with the way that an individual program is **decomposed into components**. Sommerville's Chapter 6 is mostly about this level.
- **Architecture in the large** is concerned with the architecture of **complex enterprise systems** that include other systems, programs and program components. These enterprise systems may be **distributed over different computers**, which may be owned and managed by **different companies**.

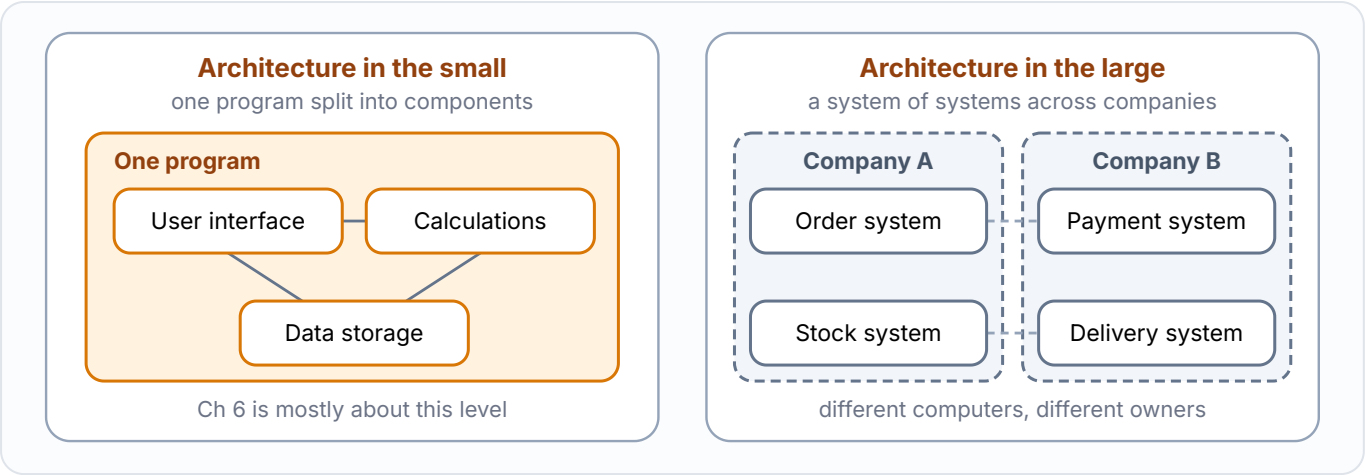


Fig 6.3 Two levels of architectural abstraction (component names are only illustrations). Drawn for this guide

6.4 Advantages of explicit architecture ★★★ MUST KNOW

"Explicit" means the architecture is deliberately designed and written down, not left to emerge by accident. Bass, Clements and Kazman suggest that explicitly designing and documenting a software architecture has **three advantages**:

Advantage	What it means
1. Stakeholder communication	The architecture is a high-level presentation of the system that may be used as a **focus of discussion** by a range of different system stakeholders.
2. System analysis	Making the architecture explicit at an early stage requires some analysis, so it becomes possible to analyse **whether the system can meet its non-functional requirements**. Architectural decisions have a profound effect on whether critical requirements such as performance, reliability and maintainability can be met.
3. Large-scale reuse	An architectural model is a compact, manageable description of how a system is organized and how its components interoperate. The architecture is often the same for systems with similar requirements, so it **may be reusable across a range of systems**. **Product-line architectures** may be developed: the same architecture is reused across a range of related systems.

MEMORY TRICK

Think **CAR**: **C**ommunication (with stakeholders), **A**nalysis (of the system), **R**euse (large-scale).

6.5 Architectural representations and their use ★★★ MUST KNOW

Box-and-line (block) diagrams ★★★ MUST KNOW

Simple, informal **block diagrams** showing entities and relationships (like Fig 6.2) are the **most frequently used** method for documenting software architectures. People from different disciplines can readily understand them. But they have been **criticised** (Bass et al.) because they lack semantics and are **very abstract**: they do not show the **types (nature) of relationships** between components, nor the **externally visible properties** of the entities or sub-systems. **However**, they are useful for **communication with stakeholders** and for **project planning**.

IN SIMPLE WORDS

Semantics = precise meaning. In Fig 6.2, is the arrow from *Vision system* to *Object identification* a function call, a message or shared data? The diagram does not say. Nor does it show what services each component offers (its "externally visible properties").

STRENGTHS

- ✓ Simple, quick and informal to draw
- ✓ Intuitive: domain experts and engineers can relate to them and join discussions
- ✓ Not cluttered with detail; show the whole system
- ✓ Managers find them helpful for project planning

WEAKNESSES

- ✗ Lack semantics (no precise meaning)
- ✗ Do not show the types of relationships between components
- ✗ Do not show externally visible properties of sub-systems
- ✗ Too abstract to be a complete design document

★ KEY POINT

Whether this criticism matters **depends on the use of architectural models**: the requirements for model semantics depend on how the models are used.

Use of architectural models ★★★ MUST KNOW

The apparent contradiction between theory ("block diagrams are poor") and industrial practice ("everyone uses them") arises because an architectural model is used in **two ways**:

	1. As a way of facilitating discussion about the system design	2. As a way of documenting an architecture that has been designed
Aim	A high-level architectural view, useful for communication with system stakeholders and project planning because it is **not cluttered with detail**.	A **complete system model** that shows the different components in a system, their **interfaces** and their **connections**.
Benefit	Stakeholders can relate to it and understand an abstract view of the system, then discuss the system as a whole without being confused by detail. It identifies the key components, so managers can start assigning people to plan their development.	A detailed description makes it easier to **understand and evolve** the system.
Notation	Informal block diagrams are enough; for many projects they are the only architectural description.	Ideally a more rigorous notation, e.g. an architectural description language (ADL, see 6.10).

A detailed, complete description leaves **less scope for misunderstanding** the relationships between components. However, it is **expensive and time-consuming** to develop, and it is practically impossible to know whether it is cost-effective, so this approach is **not widely used**.

6.6 Architectural design decisions ★★★ MUST KNOW

Architectural design is a **creative process** in which you design a system organization that will satisfy the functional and non-functional requirements of a system. There is **no formulaic** (fixed, step-by-step) architectural design process: it differs depending on the **type of system** being developed, the **background and experience** of the system architect, and the **specific requirements** for the system. So it is best to think of architectural design as a series of decisions to be made rather than a sequence of activities.

However, a number of **common decisions span all design processes**, and these decisions affect the **non-functional characteristics** of the system. Based on their knowledge and experience, architects consider the fundamental questions in Fig 6.4.

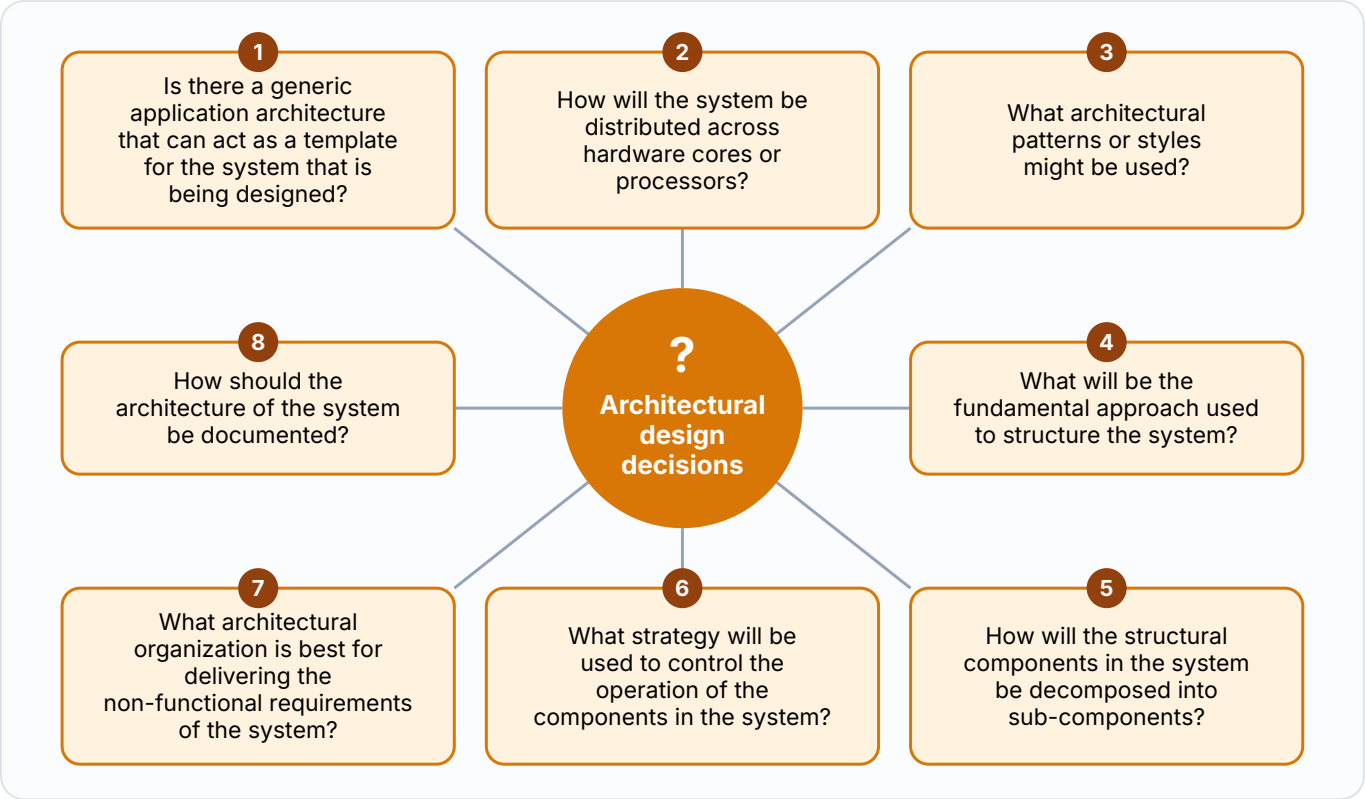


Fig 6.4 Architectural design decisions: the eight fundamental questions, numbered clockwise for revision. Sommerville Fig 6.2 · Your slides, slide 13

Notes that turn the questions into an answer:

- **Q2 (distribution)** — embedded systems and apps for PCs and mobile devices need not be distributed, but most large systems are distributed over many computers. The choice of distribution architecture is a **key decision that affects performance and reliability**.
- **Q4–Q6 (structure, decomposition, control)** — Garlan and Shaw's idea of an **architectural style** covers these three: choose the most appropriate structure (e.g. client–server or layered); decide a strategy for decomposing components into sub-components; and, in **control modelling**, develop a general model of the control relationships between the parts, i.e. how the execution of components is controlled.
- **Q1 and Q3** are about reuse (6.7); **Q7** is explained in 6.8; **Q8** (views and notations) in 6.9–6.10.
- **Evaluating** the design is difficult: the true test is how well the system meets its requirements **when in use**, but you can compare it with **reference architectures** or generic patterns.

MEMORY TRICK

Generic · **D**istribution · **P**atterns · **S**tructure · **D**ecomposition · **C**ontrol · **N**on-functional · **D**ocumentation: "**G**ood **D**evelopers **P**refer **S**imple **D**esigns; **C**lear **N**on-functional **D**ocs".

6.7 Architecture reuse ★★ IMPORTANT

Each system is unique, but architectures are often reused. When designing, you decide what your system has in common with broader classes of applications and how much knowledge from those architectures you can reuse. Three forms:

Same domain

Systems in the same application domain often have **similar architectures** that reflect domain concepts.

Product lines

Application product lines are built around a **core architecture** with **variants** that satisfy particular customer requirements.

Patterns or styles

The architecture may be designed around one or more architectural **patterns** or "styles", which capture the essence of an architecture and can be instantiated in different ways.

DEFINITION — LEARN THIS LINE

Architectural pattern (style) — a description of a system organization, such as a client–server organization or a layered architecture. It captures the essence of an architecture that has been used in different software systems. ("Pattern" and "style" now mean the same thing.)

6.8 Architecture and system characteristics ★★★ MUST KNOW

Because non-functional characteristics depend so closely on the architecture, the **choice of architectural style and structure should depend on the non-functional requirements**. For each critical requirement there is a matching architectural strategy:

If this is critical…	…use this architecture (slide wording)	How and why (textbook detail)
Performance	Localise critical operations and minimise communications. Use **large** rather than fine-grain components.	Keep critical operations in a few components on the **same computer**, not across the network. Large components reduce communications, as most interactions happen inside a component. Replicating the system on different processors may also help.
Security	Use a **layered** architecture with critical assets in the inner layers.	The most critical assets are protected in the innermost layers, with a high level of security validation applied to these layers.
Safety	**Localise** safety-critical features in a small number of sub-systems.	Co-locating safety-related operations reduces the cost of safety validation and may allow a protection system that safely shuts the system down on failure.
Availability	Include **redundant** components and mechanisms for **fault tolerance**.	Spare (redundant) components make it possible to replace and update components without stopping the system. Fault tolerance = continuing to work when a part fails.

If this is critical…	…use this architecture (slide wording)	How and why (textbook detail)
Maintainability	Use **fine-grain**, replaceable components.	Small, self-contained components can readily be changed. Separate producers of data from consumers; avoid shared data structures.

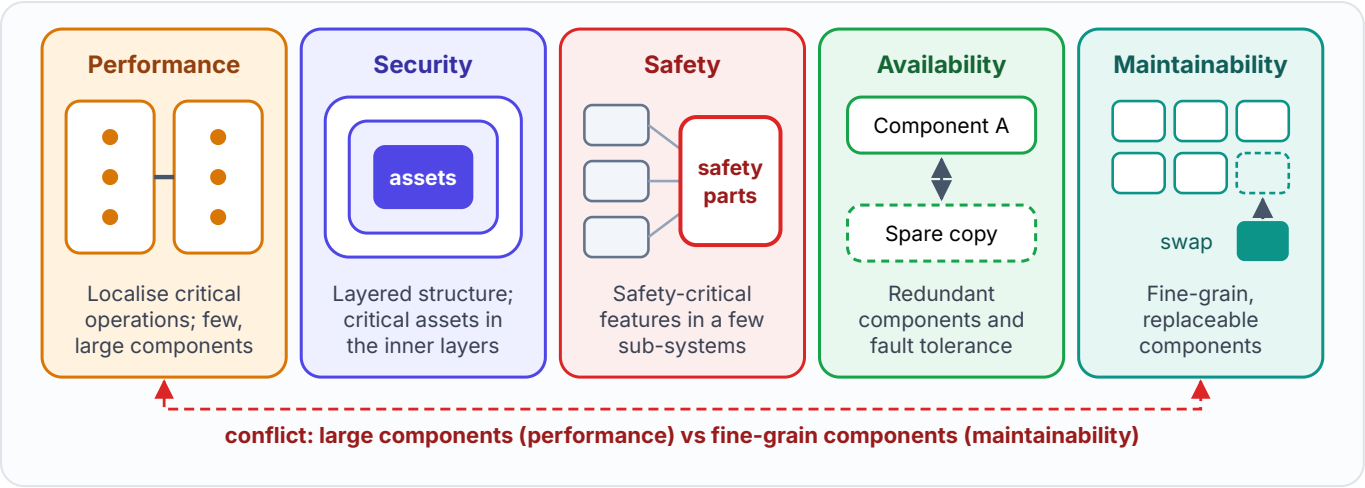


Fig 6.5 The five architectural strategies at a glance, and the textbook's example of a conflict. Drawn for this guide from Your slides, slide 15

Conflicts and trade-offs ★★★ MUST KNOW

There is potential **conflict** between some of these architectures, so a compromise must be found:

Requirements in conflict	Why they pull apart	Compromise
Performance vs maintainability (textbook example)	Performance wants a few **large** components; maintainability wants many **small, fine-grain** components.	Use **different architectural patterns or styles for separate parts** of the system.
Security vs the rest (textbook point)	Security is now almost always a critical requirement, so it cannot simply be traded away.	Design an architecture that maintains security while also satisfying the other non-functional requirements.
Security vs performance (follows from the strategies)	Layers with security validation add extra steps between components; performance wants communication minimised.	Keep time-critical operations together within one layer; apply the heaviest checks only to the inner layers.

> **EXAMPLE**
>
> In **Mentcare** (the clinics' patient information system), patient records are the critical asset, so they would sit in the innermost layer. In the **insulin pump**, the safety-critical dose computation and checks would be co-located in one small component that can be validated thoroughly.

6.9 Architectural views ★★★ MUST KNOW

Whether a model is used to focus discussion or to document a design, two questions arise (slide 17):

1. **What views or perspectives** are useful when designing and documenting a system's architecture?
2. **What notations** should be used for describing architectural models? (see 6.10)

No single diagram can show everything: **each architectural model only shows one view or perspective** of the system. It might show how a system is **decomposed into modules**, how the **run-time processes interact**, or the different ways in which components are **distributed across a network**. All of these are useful at different times, so for both design and documentation you usually need to present multiple views of the software architecture, just as a house needs a floor plan, a wiring plan and a plumbing plan.

Krutchen's 4+1 view model ★★★ MUST KNOW

In his well-known **4+1 view model** of software architecture, Krutchen suggests **four fundamental architectural views**, linked through common **use cases or scenarios** (the "+1"). A use case or scenario describes one typical interaction with the system, e.g. "a doctor records a consultation".

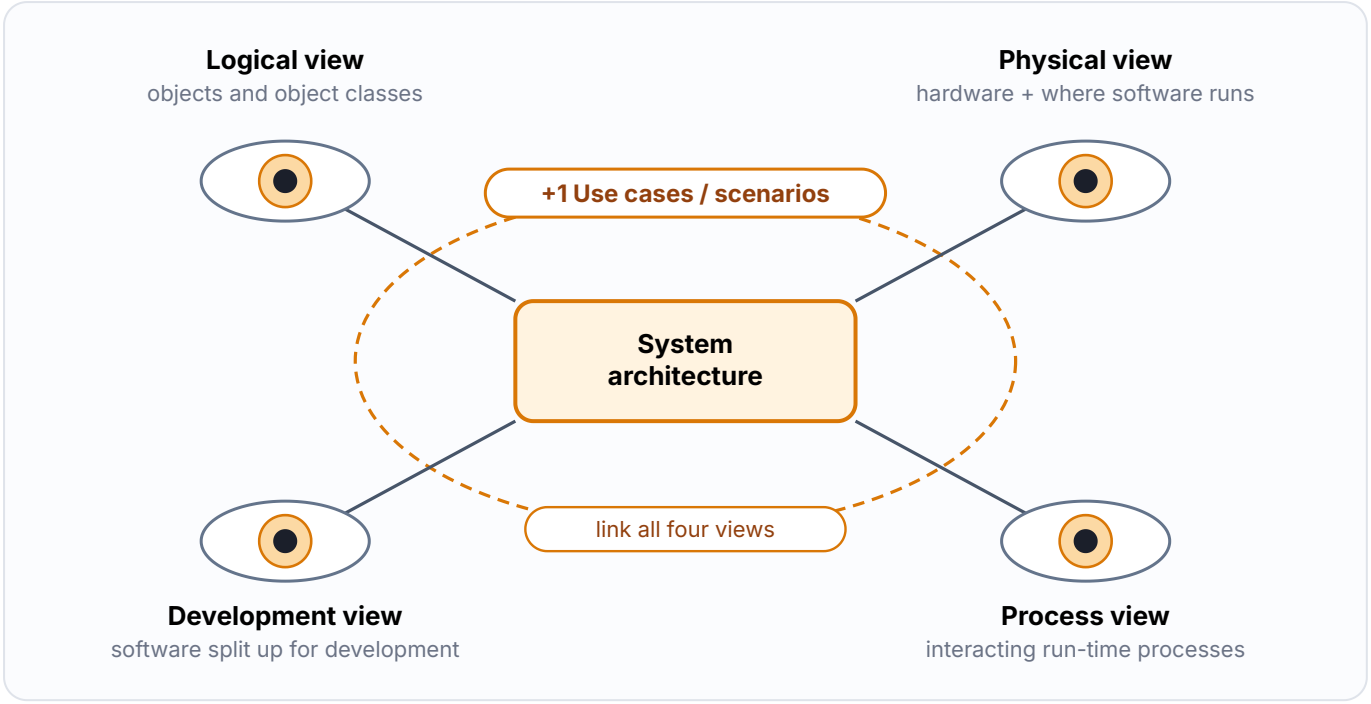


Fig 6.6 Krutchen's 4+1 view model: four views of one system architecture, linked by use cases or scenarios (+1). Sommerville Fig 6.3 · Your slides, slides 18–19 (the "+1" ring added)

View	What it shows (exam wording)	Useful for	Example: Mentcare (illustration)
Logical	The key abstractions in the system as **objects or object classes**. It should be possible to relate the system requirements to entities in this view.	Relating requirements to the design	Classes such as Patient, Consultation, Prescription, Clinic, Staff member.
Process	How, **at run-time**, the system is composed of **interacting processes**.	Judging performance, availability and other non-functional characteristics	A client process on each clinic PC talks to a server process that handles patient-record requests.
Development	How the software is **decomposed for development**: its breakdown into components implemented by a single developer or development team.	Software managers and programmers	Modules for patient records, appointments, prescribing and reporting, each given to one team.

View	What it shows (exam wording)	Useful for	Example: Mentcare (illustration)
Physical	The **system hardware** and how software components are **distributed across the processors** in the system.	Systems engineers planning a system deployment	Clinic PCs connected over the network to a central database server.
+1 Scenarios	Relate the four views to each other.	Linking and checking the views	"Doctor records a consultation" traced through all four views.

⚠ **WATCH OUT**

Process view (run-time processes) ≠ development view (code split among teams); physical view (hardware) ≠ logical view (classes). The "+1" links the **four** views; it is not a fifth view.

✎ **EXAM TIP**

10M: draw Fig 6.6 → one line per view (shows + useful for) → the +1 → a Mentcare example each.

The conceptual view, and how many views are needed ★★ IMPORTANT

Hofmeister et al. suggest similar views but add a **conceptual view**: an abstract view of the system that can (1) be the basis for decomposing high-level requirements into more detailed specifications, (2) help engineers make decisions about components that can be reused, and (3) represent a **product line** rather than a single system. **The packing robot architecture (Fig 6.2) is an example of a conceptual view.**

In practice, conceptual views are almost always developed during design, to explain the architecture to stakeholders and inform decision making. Other views may be developed as different aspects are discussed, but it is **rarely necessary to develop a complete description from all perspectives**. Architectural patterns can also be associated with the different views.

6.10 Representing architectural views ★★ IMPORTANT

Which **notation** should describe the views: the **UML** (Unified Modeling Language, the standard diagrams for object-oriented software) or **ADLs** (architectural description languages)?

- Some people argue that the **UML** is an appropriate notation for describing and documenting system architectures.
- **Sommerville disagrees** (he is the "I" on slide 20): the UML does not include abstractions appropriate for **high-level system description**; it was designed for object-oriented systems, and object classes are too close to the implementation. He prefers **informal notations**, quicker to write and easy to draw on a whiteboard. The UML is most valuable for **documenting an architecture in detail** or for **model-driven development** (where models are used to generate the system).
- **ADLs** have been developed but are **not widely used**. Their basic elements are **components and connectors**, with rules for well-formed architectures. They are precise, but specialist languages that domain experts find hard to use.

★ **KEY POINT**

Informal models and notations, such as the UML, will remain the most common. Agile users say detailed design documents are mostly unused; Sommerville largely agrees. **Except for critical systems**, do not develop a detailed description from all four of Krutchen's views: develop the views **useful for communication** and do not worry about completeness.

Chapter 6 in one page

Definitions

- **Architectural design**: understanding how a system should be organized and designing its overall structure. It is the critical link between requirements engineering and design (it identifies the main components and their relationships). **Output**: an architectural model showing the system as a set of communicating components.
- **In the small**: one program decomposed into components. **In the large**: enterprise systems of systems, programs and components on computers that may belong to different companies.
- **Pattern/style**: a description of a system organization (client–server, layered).
- **Product line**: a core architecture plus variants for particular customers.
- **Conceptual view**: an abstract view used to decompose requirements, decide on reuse and represent product lines (e.g. the packing robot).

Agility and architecture

Design the architecture early: refactoring components is easy, but refactoring the architecture is expensive because it affects most components.

Advantages of explicit architecture (CAR)

1. Stakeholder communication
2. System analysis: can the non-functional requirements be met?
3. Large-scale reuse, including product-line architectures

Box-and-line diagrams

+ simple, intuitive, good for stakeholders and planning. **–** lack semantics; do not show relationship types or visible properties.
Two uses of models: (1) facilitating discussion: high-level, uncluttered; (2) documenting a designed architecture: complete, with components, interfaces and connections.

8 design decisions (Good Developers Prefer Simple Designs; Clear Non-functional Docs)

1. Generic application architecture as a template?
2. Distribution across cores/processors?
3. Patterns or styles?
4. Fundamental approach to structure?
5. Decomposition into sub-components?
6. Control strategy?
7. Organization for non-functional requirements?
8. How to document?

Non-functional requirement → architecture

- **Performance**: localise critical operations; few, large components
- **Security**: layered, with critical assets in the inner layers
- **Safety**: safety features in a small number of sub-systems
- **Availability**: redundant components, fault tolerance
- **Maintainability**: fine-grain, replaceable components

Conflict: performance (large) vs maintainability (fine-grain). Compromise: use different styles for different parts.

4+1 views (Krutchen)

Logical: objects/classes · **Process**: run-time processes · **Development**: decomposition for teams · **Physical**: hardware and distribution · **+1**: use cases/scenarios link them.

Notations

UML: too low-level for high-level design (Sommerville); best for detailed documentation. ADLs: components + connectors; precise but not widely used. Informal notations are enough except for critical systems.

Diagrams to practise

Fig 6.2 packing robot · Fig 6.4 design decisions · Fig 6.6 4+1 views

Likely exam questions

2M **Define architectural design. Why is it called the critical link between design and requirements engineering?**

How to answer → Definition (6.1) → identifies the main components and their relationships → output: an architectural model.

4M **Explain the relationship between agile development and architecture.**

How to answer → Designed early even in agile; refactoring the architecture is expensive (affects most components).

4M **Differentiate architecture in the small and architecture in the large.**

How to answer → Two definitions (6.3), one example each; sketch Fig 6.3.

6M **What are the advantages of explicitly designing and documenting a software architecture?**

How to answer → CAR: stakeholder communication, system analysis, large-scale reuse (6.4).

8M **With a neat diagram, explain the architecture of a packing robot control system.**

How to answer → What the robot does → Fig 6.2 → boxes, boxes in boxes, arrows → a conceptual view.

6M **"Box-and-line diagrams are poor architectural representations." Discuss, and explain the two uses of architectural models.**

How to answer → Criticism → strengths → the two uses as a table → semantics needed depend on use (6.5).

10M **Explain architectural design decisions with a neat diagram.**

How to answer → Creative process (3 factors) → series of decisions → Fig 6.4, all 8 questions → notes on Q2 and Q4–Q6.

8M **How do non-functional requirements affect the architecture? Give the strategy for performance, security, safety, availability and maintainability.**

How to answer → Bosch point → 5-row table (6.8) → performance vs maintainability conflict and compromise.

4M **Explain architecture reuse.**

How to answer → Same domain; product lines (core + variants); patterns/styles with definition (6.7).

10M **Explain Krutchen's 4+1 view model with a neat diagram. Illustrate the views for a system such as Mentcare.**

How to answer → Why multiple views → Fig 6.6 → four views + "useful for" → +1 → Mentcare column.

5M **Discuss the notations used to represent architectural views. Is the UML suitable?**

How to answer → For vs against the UML → ADLs → informal notations enough except for critical systems (6.10).

2M **What is a conceptual view? Give an example.**

How to answer → Hofmeister et al.; three uses; the packing robot (Fig 6.2).

Architectural patterns (MVC, layered, repository, client–server, pipe and filter) and application architectures complete Chapter 6 and are studied after CIE-1.

EXAM DAY

Exam-Day Playbook

How to turn what you studied into marks: answer structure, command words, diagrams and the comparison questions to expect.

Writing answers that score

2 marks

- One-line **definition** in textbook words
- Plus **one** example or key feature
- 2–3 lines total. Don't write an essay.

4–6 marks

- Definition
- A small diagram or a table, if one exists
- 4–6 points, each with a bold keyword and one explaining line
- One example

8–10 marks

- Definition / introduction (2 lines)
- **Neat labelled diagram**, drawn first
- Explain each box or phase under its own sub-heading
- Advantages and disadvantages, or a comparison
- When to use it + a case-study example
- One-line conclusion

★ KEY POINT

Examiners mark **keywords**, **structure** and **diagrams**. Underline the keywords (the highlighted phrases in this guide), number your points, and leave space between them.

Diagram rules

1. **Draw first, write after.** The diagram tells you what to explain, and it scores even if time runs out.
2. **Label every box and arrow** using the textbook's names (for example "Requirements definition", not "Req").
3. **Give it a caption**: *Fig: The waterfall model.*
4. **Use boxes for activities, arrows for flow**, and dashed arrows for feedback or iteration.
5. **Keep it big and clean.** Half a page is fine for a 10-mark question.

Command words decoded

If the question says…	…it wants
Define / What is	The exact definition, plus one example. Short.
List / Enumerate	The complete list, numbered. One line each is enough.

If the question says…	…it wants
Explain / Describe	Definition + points with explanation + diagram if one exists + example.
With a neat diagram	The diagram carries a large share of the marks. Draw it carefully and explain every part.
Differentiate / Compare / Distinguish	A **table** with a "Basis" column and 5–7 rows. Add one line each of "use A when…, use B when…".
Discuss / Justify	Points for and against, then your conclusion with a reason.
Apply to / For the given case study	Name the system in every point. Generic answers lose marks here.

✎ **CASE-STUDY QUESTIONS**

When a question names a system (insulin pump, Mentcare, weather station, iLearn, or something new like a library or ATM system), connect every point to it: "*Because the insulin pump is safety-critical, …*". If the system is new, pick the textbook system it most resembles and reuse that reasoning.

Diagrams you must be able to draw

Every textbook or slide diagram in this guide. Tick each one when you can draw it from memory with all labels. Diagrams drawn only to help understanding are not listed.

Ch 1 · Introduction

☐ **Fig 1.3** The four essential attributes of good software

☐ **Fig 1.4** Software engineering is one part of system engineering and builds on computer science

☐ **Fig 1.5** Insulin pump hardware architecture

☐ **Fig 1.6** Activity model of the insulin pump (UML)

☐ **Fig 1.7** Organisation of the Mentcare system

☐ **Fig 1.8** The weather station's environment (UML packages with the «system» stereotype)

☐ **Fig 1.9** The iLearn architecture

Ch 2 · Software Processes

☐ **Fig 2.1** The waterfall model

☐ **Fig 2.2** Incremental development

☐ **Fig 2.3** Reuse-oriented software engineering

☐ **Fig 2.4** The V-model as drawn by Pressman

☐ **Fig 2.5** The prototyping paradigm

☐ **Fig 2.6** A typical spiral model

☐ **Fig 2.7** The requirements engineering process

☐ **Fig 2.8** A general model of the design process

☐ **Fig 2.9** Stages of testing (slide 28 labels the last stage "Acceptance testing")

☐ **Fig 2.10** Testing phases in a plan-driven software process (V-model)

☐ **Fig 2.11** System evolution

☐ **Fig 2.12** The process of prototype development

Ch 4 · Requirements Engineering

☐ **Fig 4.1** User and system requirements

☐ **Fig 4.2** Readers of different types of requirements specification

☐ **Fig 4.3** The eight stakeholder groups of the Mentcare system (the textbook also includes…

☐ **Fig 4.4** One non-functional requirement can generate new functional requirements and restrict…

☐ **Fig 4.5** Types of non-functional requirement

☐ **Fig 4.7** A spiral view of the requirements engineering process

☐ **Fig 4.8** The requirements elicitation and analysis process (step 4 is called "requirements…

☐ **Fig 4.9** Ethnography and prototyping for requirements analysis

☐ **Fig 4.10** Use cases for the Mentcare system

☐ **Fig 4.11** Users of a requirements document and how they use it

☐ **Fig 4.12** Requirements evolution

☐ **Fig 4.13** Requirements change management

☐ **Fig 4.14** What the insulin pump software does, with the textbook requirements that govern each step

Ch 5 · Design Concepts

☐ **Fig 5.1** Design is the last action of modeling

☐ **Fig 5.2** Translating the requirements model into the design model

☐ **Fig 2.13** Incremental delivery

☐ **Fig 2.14** The process improvement cycle

☐ **Fig 2.15** Capability maturity levels as a staircase (box names from slide 54; grey meanings from...

Ch 3 · Agile

☐ **Fig 3.1** Plan-driven and agile development

☐ **Fig 3.3** The XP release cycle

☐ **Fig 3.4** A "prescribing medication" story card

☐ **Fig 3.5** Examples of task cards for prescribing medication (Task 3 shown in full)

☐ **Fig 3.8** Test case description for dose checking

☐ **Fig 3.9** The Scrum sprint cycle

☐ **Fig 3.11** Distributed Scrum

☐ **Fig 3.12** Factors influencing the choice of plan-based or agile development

☐ **Fig 5.3** The textbook's door example

☐ **Fig 5.4** Modularity and software cost

☐ **Fig 5.8** A crosscutting concern (user validation) as scattered/tangled fragments (left) versus...

☐ **Fig 5.9** The textbook's refactoring case

☐ **Fig 5.10** Core OO ideas

☐ **Fig 5.11** The analysis class FloorPlan refined into a design class

☐ **Fig 5.13** Dimensions of the design model

☐ **Fig 5.14** Interface representation for ControlPanel

☐ **Fig 5.15** (a) UML component diagram

Ch 6 · Architectural Design

☐ **Fig 6.2** The architecture of a packing robot control system

☐ **Fig 6.4** Architectural design decisions

☐ **Fig 6.5** The five architectural strategies at a glance, and the textbook's example of a conflict

☐ **Fig 6.6** Krutchen's 4+1 view model